FIAP - ANÁLISE E DESENVOLVIMENTO DE SISTEMAS
DOMAIN DRIVEN DESIGN USING JAVA

SoulUp

Documentação da API RESTful - Sprint 4

Equipe: SoulUp
Turma: 1TDSPO

Matheus Ruiz Giatti - RM 570701
Matheus Leite Souza das Virgens - RM 573416

São Paulo, novembro de 2026

Sumário

Item	Seção
1	Objetivo e escopo do projeto
2	Funcionalidades implementadas
3	Arquitetura da aplicação
4	Tabela de endpoints (API RESTful)
5	Protótipo: prints das telas implementadas
6	Modelo de Entidade-Relacionamento (MER)
7	Diagrama de classes
8	Procedimentos para rodar a aplicação
9	Links da entrega

1. Objetivo e escopo do projeto

O SoulUp é uma plataforma que transforma ações sustentáveis do dia a dia em pontos, níveis e recompensas, com o avatar Soul acompanhando a evolução do usuário. Nesta sprint, o módulo Java, que na Sprint 3 rodava em console com uma camada DAO em JDBC, passou a ser uma **API RESTful em Spring Boot**, publicada na nuvem e consumida pelo front-end em React.

A API recebe e devolve dados em JSON, aplica as regras de negócio do SoulUp e grava tudo no banco Oracle modelado em Building Relational Database. É o mesmo esquema usado pelo sistema em Python, então o que é cadastrado em um sistema aparece no outro.

Escopo: contas de usuário (com o avatar Soul), catálogo de ações sustentáveis, registro de ações realizadas com cálculo de pontos, nível e impacto, e missões com conclusão automática. **Fora do escopo:** autenticação por senha, resgate de recompensas pela API e respostas do avatar geradas por inteligência artificial. As tabelas de recompensas, resgates e interações estão mapeadas como entidades, mas não possuem endpoints nesta entrega.

2. Funcionalidades implementadas

- **CRUD de usuários:** cadastro, listagem em ordem de ranking, consulta, alteração de nome e e-mail e exclusão. O cadastro já cria o avatar Soul e inicia o usuário no nível 1, sem pontos.
- **E-mail único:** a API recusa um e-mail já cadastrado, sem diferenciar maiúsculas e minúsculas (resposta 409).
- **Catálogo de ações:** lista as ações sustentáveis com categoria e pontuação padrão.
- **CRUD de registros de ação:** registrar, consultar, corrigir e excluir uma ação realizada. A pontuação vem do catálogo e o impacto (MINIMO, BAIXO, MEDIO ou ALTO) é calculado a partir dela.
- **Pontos e nível:** cada registro soma experiência e saldo. O nível é recalculado pelas faixas de 50, 200, 500 e 1000 pontos de experiência. Corrigir ou excluir um registro ajusta os pontos pela diferença, e a API impede retirar pontos que já foram gastos em resgates.

- **Missões automáticas:** ao registrar uma ação, a API verifica as metas (quantidade de ações, categorias diferentes, experiência ou nível), conclui as missões atingidas e credita o bônus uma única vez, na mesma transação.
- **Histórico e progresso:** histórico de ações de cada usuário e a lista de missões com status e progresso até a meta.
- **Validação e erros:** Bean Validation nos dados de entrada (400), recurso inexistente (404), conflito de regra (409), sempre com mensagem em português no corpo da resposta.
- **Integração com o front-end:** classe `CorsFilter` liberando as requisições do site publicado na Vercel, e documentação interativa com Swagger.

3. Arquitetura da aplicação

O projeto segue a organização em camadas vista em aula (Controller, Service, Repository e Model), com DTOs em `record` para entrada e saída de dados:

Pacote	Camada	Responsabilidade
`model`	Modelo (entities)	11 classes: uma entidade JPA para cada uma das 10 tabelas, mais a chave composta `UsuarioMissaoId`.
`repository`	DAO	Interfaces que estendem `JpaRepository`: inserir, consultar, alterar e excluir sem escrever SQL manual, além das consultas específicas (ranking, histórico, contagens).
`service`	BO (regras de negócio)	Validações e regras: e-mail único, nível, impacto, pontos, missões e exclusão em cascata. Métodos de escrita são transacionais.
`dto`	Transferência de dados	Records de requisição (com Bean Validation) e de resposta; a entidade nunca é exposta diretamente.
`controller`	Resource	Endpoints REST com os verbos e status HTTP corretos, documentados com Swagger.
`config`	Configuração	`CorsFilter`: cabeçalhos de CORS e resposta ao preflight (OPTIONS).

Conexão com o banco. No Spring Boot, a fábrica de conexões é o `DataSource` que o framework cria a partir do `application.properties` (URL do Oracle, driver ojdbc11, usuário e senha). Ele substitui a classe `ConnectionFactory` da Sprint 3 e mantém um pool de conexões (HikariCP). Usuário e senha são lidos de variáveis de ambiente (`DB_USER` e `DB_PASSWORD`), para não ficarem gravados no repositório público.

Identificadores. Os IDs de usuário, avatar, registro, resgate e interação são gerados pelas sequências do Oracle (`SQ_SU_USUARIO`, `SQ_SU_REGISTRO` e outras), as mesmas usadas pelo sistema em Python. A API não cria nem altera tabelas (`ddl-auto=none`): o esquema é o do script DDL de Building Relational Database.

Tratamento de exceções. As regras de negócio lançam `ResponseStatusException` com o status e a mensagem adequados, e os erros de validação do `@Valid` retornam 400 com o campo e a mensagem de cada problema. O front-end exibe essas mensagens ao usuário.

Exemplo de resposta de erro

```
GET /usuarios/99999  ->  404 Not Found
{
  "status": 404,
  "error": "Not Found",
  "message": "Usuário não encontrado com o ID 99999",
  "path": "/usuarios/99999"
}
```

4. Tabela de endpoints (API RESTful)

URL base: `https://soulup-api-k9nq.onrender.com`

Verbo	URI	Descrição	Status de resposta
GET	`/`	Verifica se a API está no ar	200
GET	`/usuarios`	Lista os usuários em ordem de ranking (maior experiência primeiro)	200, 500
GET	`/usuarios/{id}`	Busca um usuário	200, 404, 500
POST	`/usuarios`	Cadastra o usuário e cria o avatar Soul	201, 400, 409, 500
PUT	`/usuarios/{id}`	Atualiza nome e e-mail	200, 400, 404, 409, 500
DELETE	`/usuarios/{id}`	Exclui o usuário, o avatar e o histórico dele	204, 404, 500
GET	`/usuarios/{id}/registros`	Histórico de ações do usuário	200, 404, 500
GET	`/usuarios/{id}/missoes`	Missões com status, progresso e bônus do usuário	200, 404, 500
GET	`/acoes`	Catálogo de ações sustentáveis com categoria e pontos	200, 500
GET	`/missoes`	Catálogo de missões com critério e meta	200, 500
GET	`/registros/{id}`	Busca um registro de ação	200, 404, 500
POST	`/registros`	Registra uma ação, soma os pontos, recalcula o nível e conclui missões	201, 400, 404, 500
PUT	`/registros/{id}`	Corrige a ação ou a descrição e ajusta os pontos pela diferença	200, 400, 404, 409, 500
DELETE	`/registros/{id}`	Exclui o registro e retira os pontos da ação	204, 404, 409, 500

200 OK · 201 Created · 204 No Content · 400 Bad Request (dados inválidos) · 404 Not Found (recurso inexistente) · 409 Conflict (e-mail repetido ou pontos já utilizados) · 500 Internal Server Error (falha inesperada, como perda de conexão com o banco).

Exemplo real: POST /registros

Ação de 70 pontos que concluiu a missão "Primeira Acao Verde" (+20 de bônus): o usuário de teste foi de 0 para 90 pontos e subiu para o nível 2.

```
Requisição
{ "idUsuario": 12, "idAcao": 4, "descricao": "Levei um monitor antigo ao ponto de coleta" }

Resposta 201 Created
{
  "registro": { "id": 27, "nomeAcao": "Descartar eletronico em ponto de coleta",
                "categoria": "Reciclagem", "pontuacao": 70, "impacto": "MEDIO", ... },
  "usuario":  { "id": 12, "pontosExperiencia": 90, "saldoPontos": 90, "nivel": 2, ... },
  "missoesConcluidas": ["Primeira Acao Verde"]
}   (trechos resumidos com "...")
```

5. Protótipo: prints das telas implementadas

As telas abaixo são capturas reais do front-end em React consumindo esta API, com os dados do banco Oracle. Cada tela indica os endpoints que utiliza.

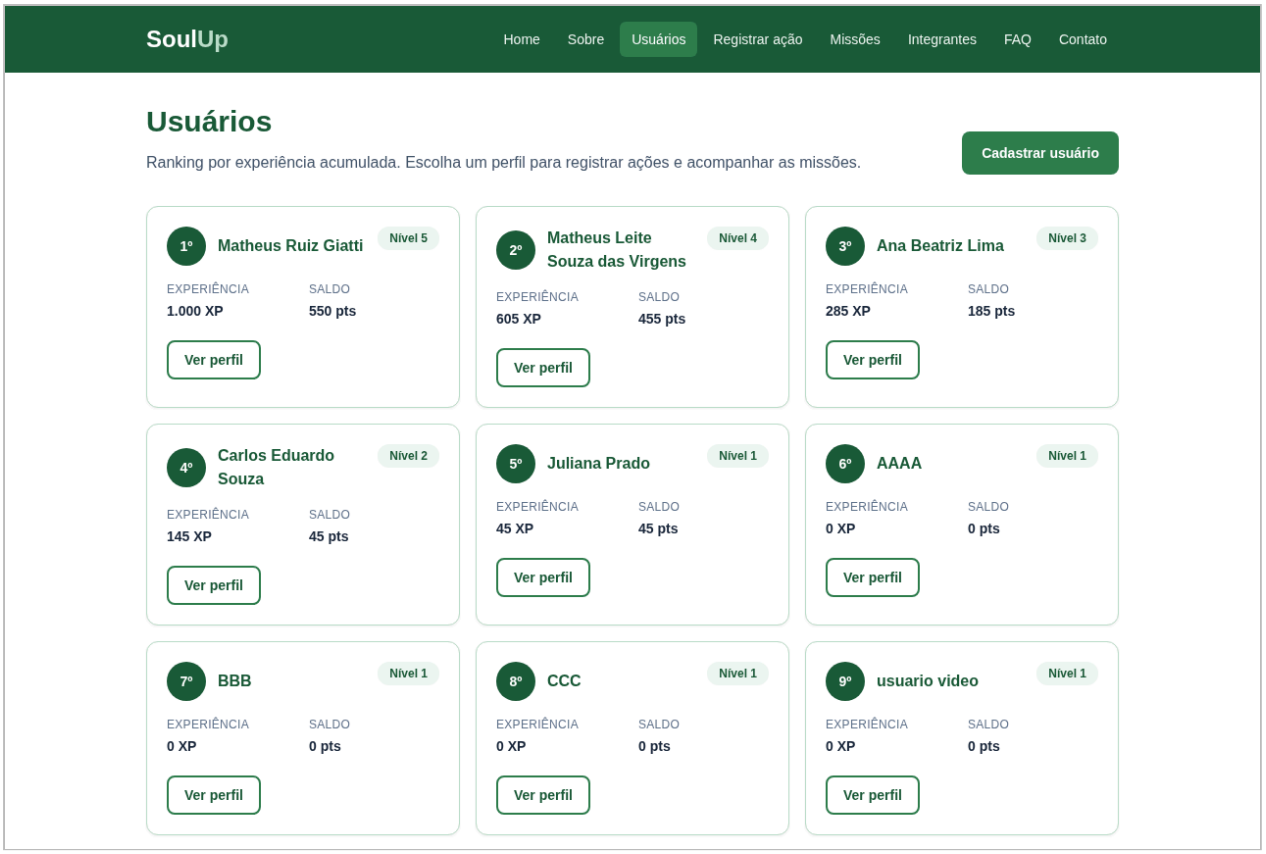


Figura 1 - Usuários: ranking por experiência (GET /usuarios). O botão "Cadastrar usuário" abre o formulário do POST /usuarios.

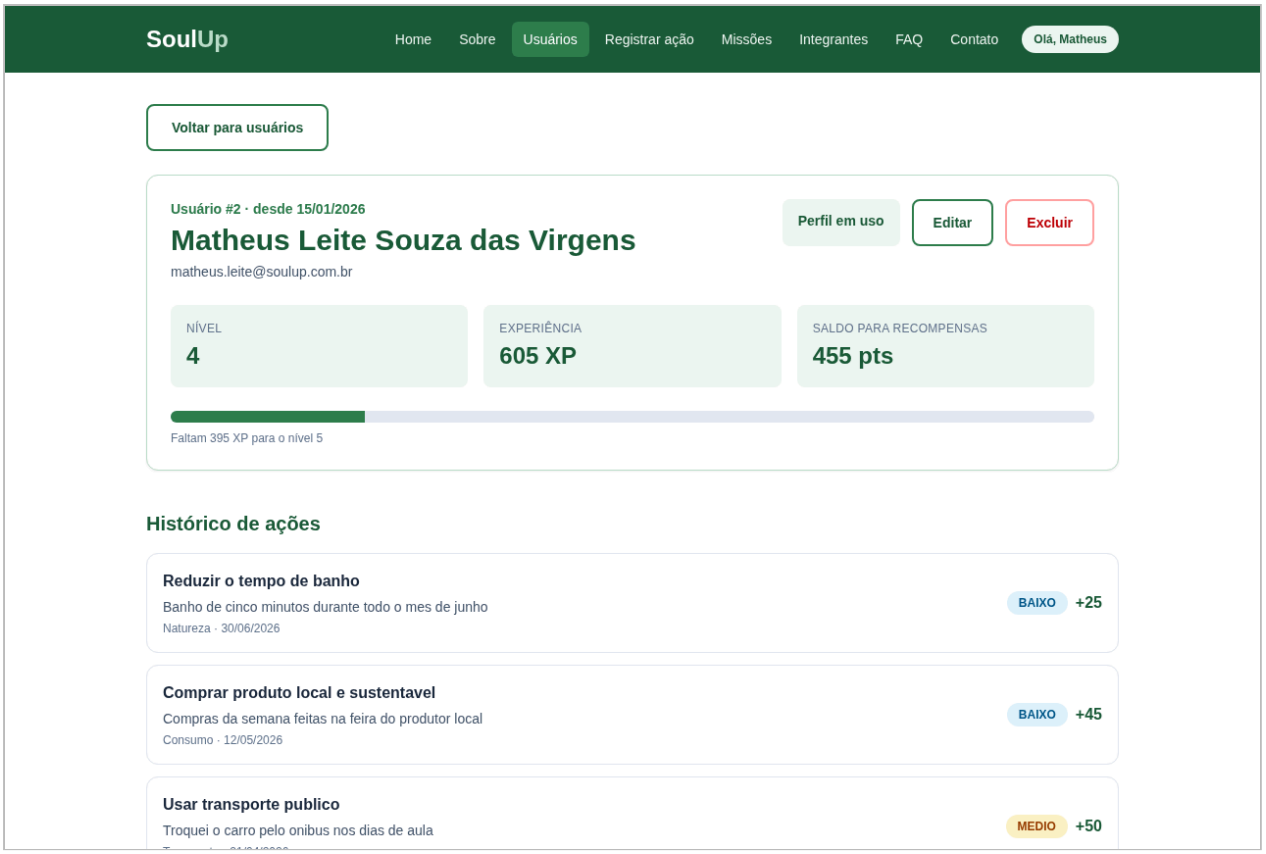


Figura 2 - Perfil do usuário: nível, experiência, saldo e histórico (GET /usuarios/{id} e GET /usuarios/{id}/registros). Os botões Editar e Excluir usam PUT e DELETE /usuarios/{id}.

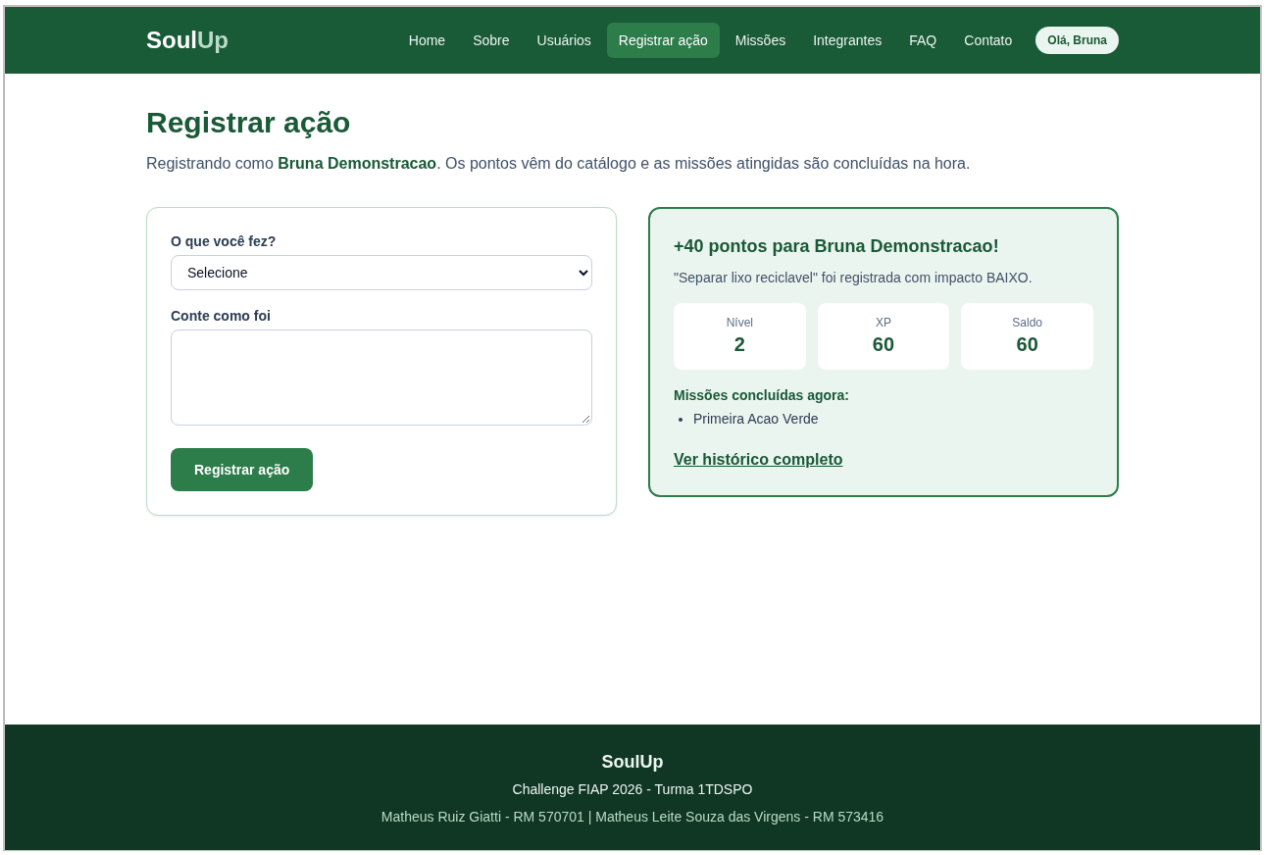


Figura 3 - Registrar ação: o catálogo vem do GET /acoes e o envio faz o POST /registros. A resposta mostra os pontos ganhos, o novo nível e a missão concluída na mesma operação.

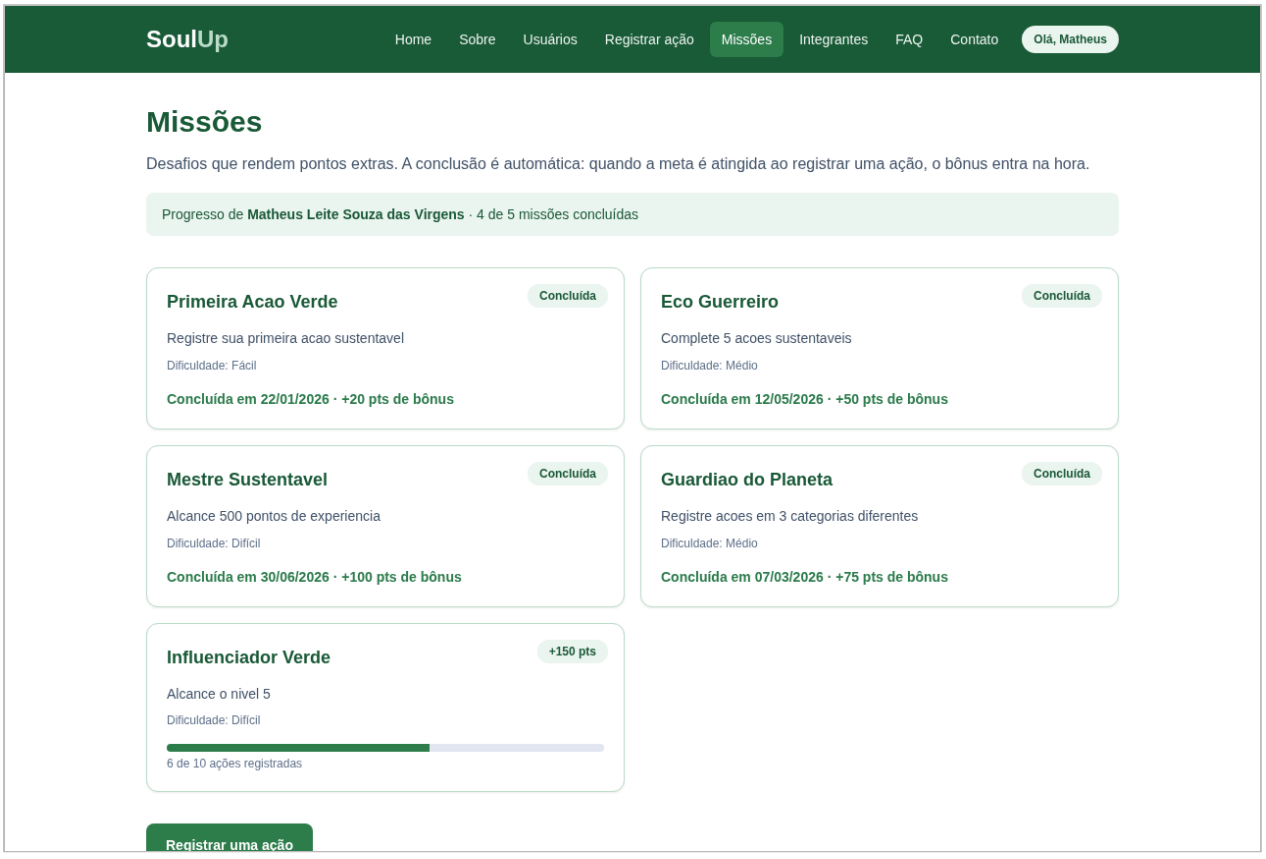


Figura 4 - Missões do usuário em uso, com status e progresso calculados pela API (GET /usuarios/{id}/missoes).

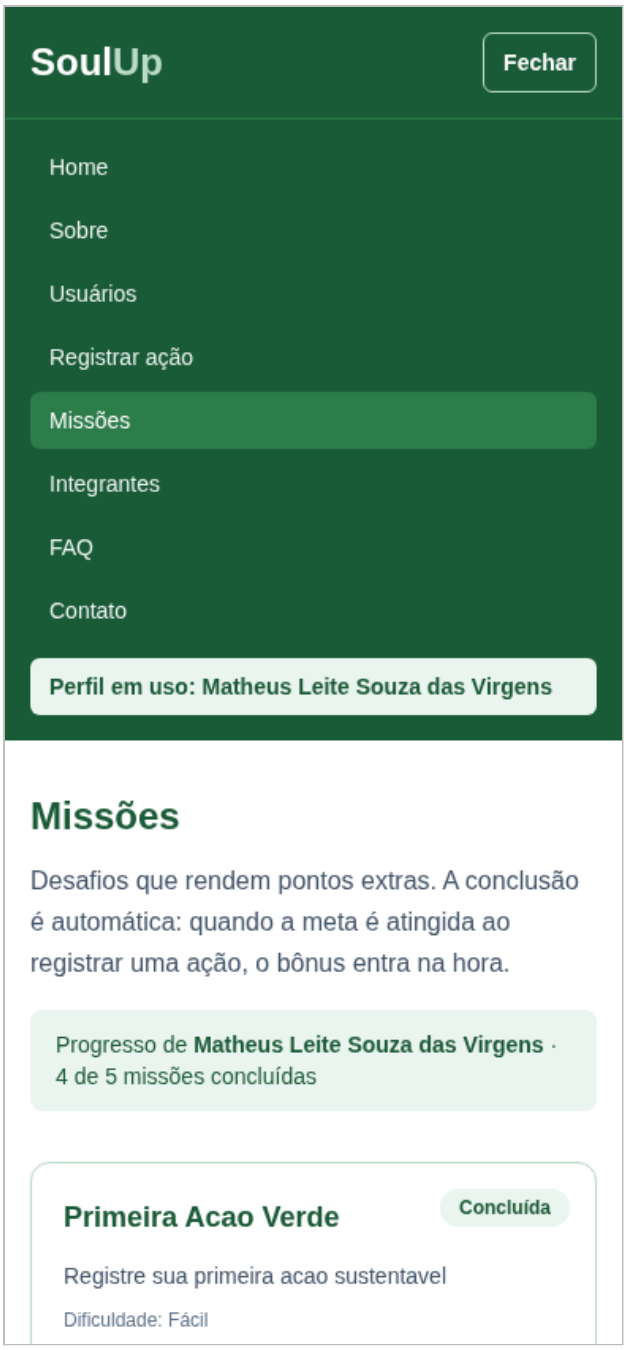


Figura 5 - Versão para celular: menu e missões.

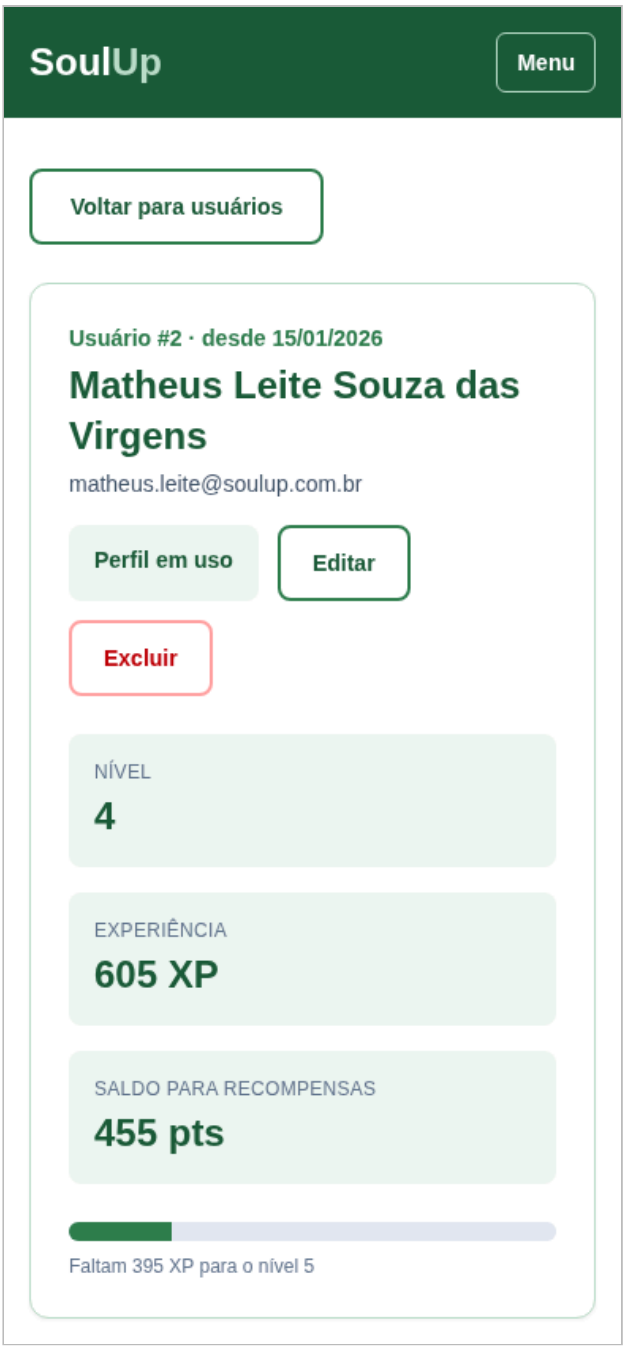


Figura 6 - Versão para celular: perfil do usuário.

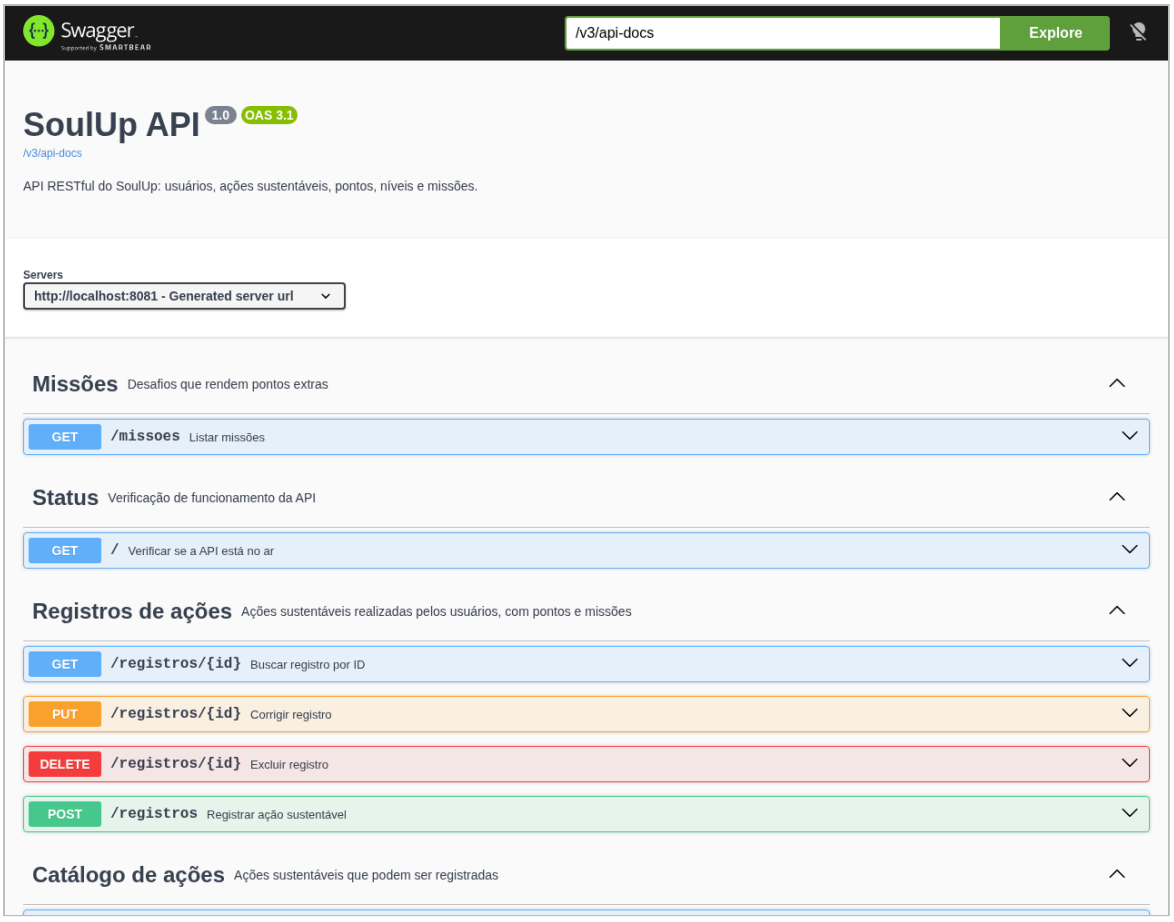


Figura 7 - Documentação interativa da API (Swagger UI), com os endpoints agrupados por recurso.

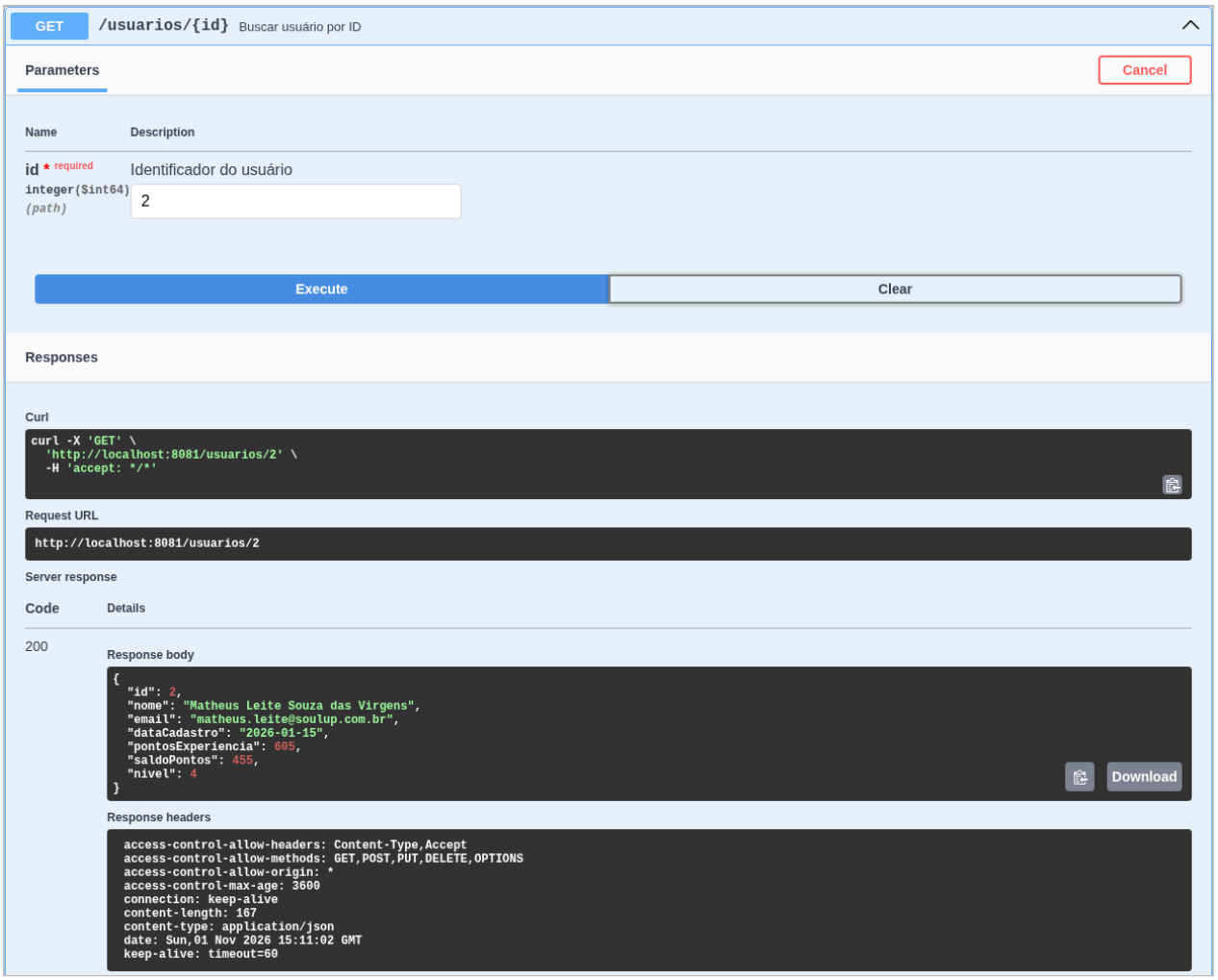


Figura 8 - Execução do GET /usuarios/{id} pelo Swagger (execução local da mesma versão publicada): resposta 200 com o JSON do usuário e os cabeçalhos de CORS adicionados pelo CorsFilter.

6. Modelo de Entidade-Relacionamento (MER)

Banco Oracle com dez tabelas: quatro de catálogo (categoria, ação, missão e recompensa), duas de conta (usuário e avatar) e quatro de registro (registro de ação, usuário-missão, resgate e interação). PK indica chave primária, FK chave estrangeira e UK restrição UNIQUE. As colunas `ds_tipo_criterio`, `qt_meta` e `qt_bonus_concedido` foram acrescentadas na Sprint 4 para a conclusão automática das missões.

Relacionamento	Cardinalidade
Categoria classifica Ação	1 : N
Usuário possui Avatar	1 : 1 (id_usuario UNIQUE em T_SU_AVATAR)
Usuário realiza Registro de ação / Ação é registrada em Registro de ação	1 : N / 1 : N
Usuário participa de Missão (tabela associativa T_SU_USUARIO_MISSAO)	N : N
Usuário faz Resgate / Recompensa é resgatada em Resgate	1 : N / 1 : N
Avatar conversa em Interação	1 : N

O diagrama completo está na próxima página, girado para caber no tamanho da folha.

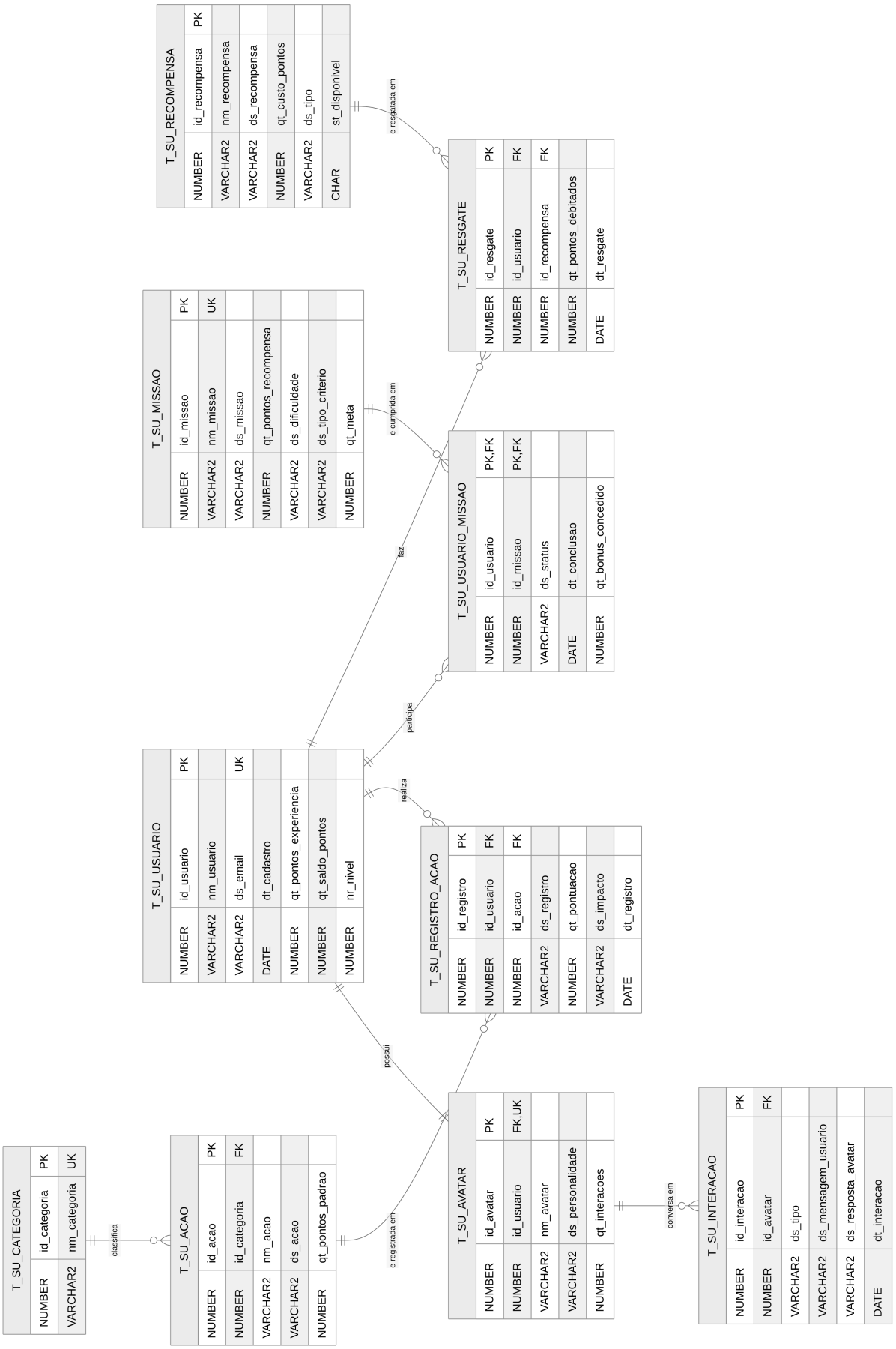


Figura 9 - MER do SoulUp (modelo físico no Oracle).

7. Diagrama de classes

7.1 Camada de modelo (entidades JPA)

Cada entidade corresponde a uma tabela do MER. As chaves estrangeiras viram associações (`@ManyToOne` e `@OneToOne`), e a tabela associativa T_SU_USUARIO_MISSAO usa a chave composta `UsuarioMissaoId` (`@EmbeddedId`). Todas as entidades possuem construtor vazio, construtor completo, getters e setters.

O diagrama está na próxima página, girado para caber no tamanho da folha.

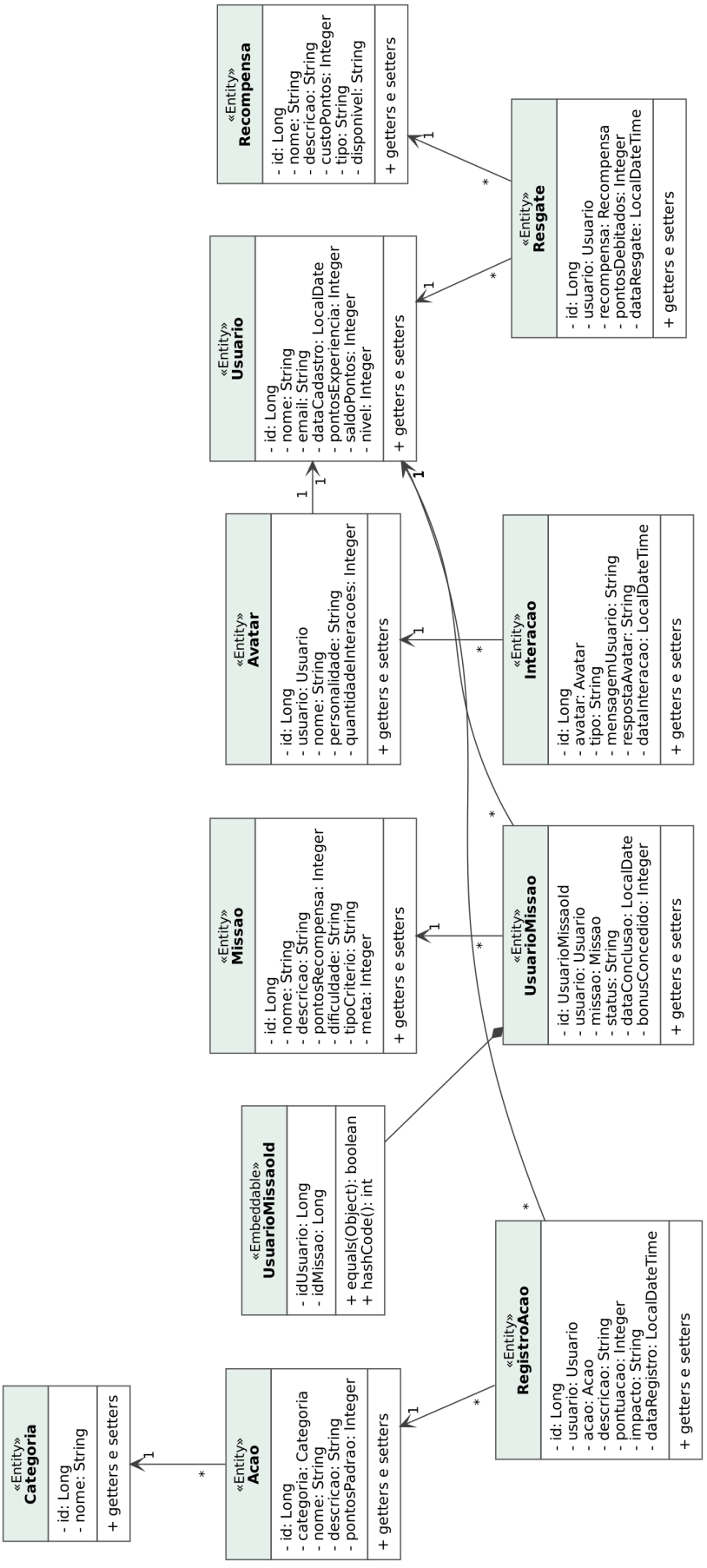


Figura 10 - Diagrama de classes da camada de modelo.

7.2 Camadas Controller, Service e Repository

Setas tracejadas indicam dependência (injeção pelo construtor) e setas com triângulo vazio indicam herança de interface. Os controllers dependem apenas dos services, que concentram as regras de negócio e usam os repositories para acessar o banco.

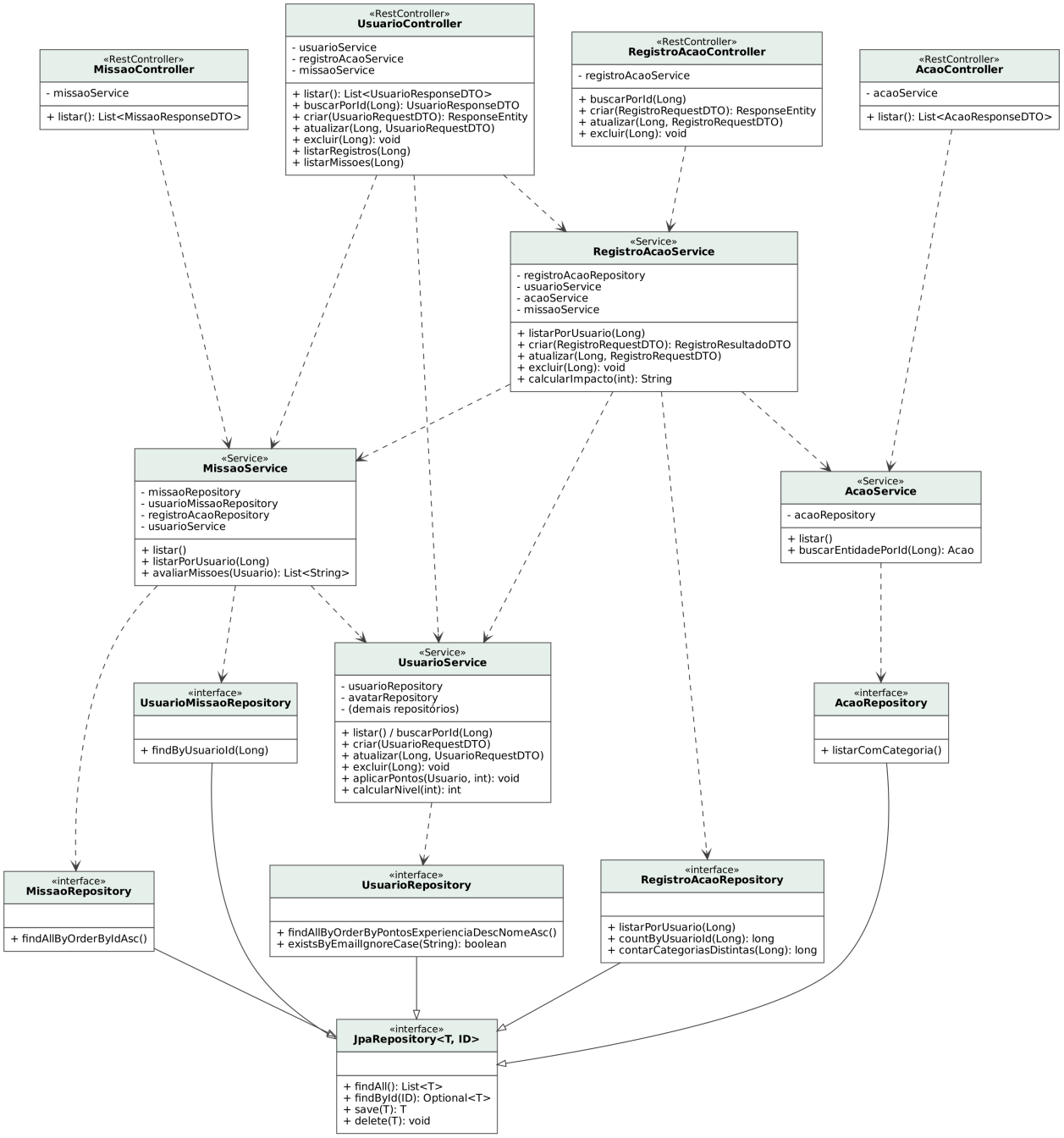


Figura 11 - Diagrama de classes das camadas de serviço e acesso a dados.

7.3 Resumo das classes

Classe	Pacote	Papel
Usuario, Avatar, Categoria, Acao, Missao, Recompensa, RegistroAcao, UsuarioMissao, UsuarioMissaoId, Resgate, Interacao	model	Entidades mapeadas para as 10 tabelas
UsuarioRepository, AvatarRepository, AcaoRepository, MissaoRepository, RegistroAcaoRepository, UsuarioMissaoRepository, ResgateRepository, InteracaoRepository	repository	Acesso a dados (JpaRepository)
UsuarioService	service	Cadastro com avatar, e-mail único, exclusão em cascata, `aplicarPontos` e `calcularNivel`
RegistroAcaoService	service	Registro, correção e exclusão de ações; `calcularImpacto`
MissaoService	service	Progresso das missões e `avaliarMissoes` (conclusão automática com bônus)
AcaoService	service	Catálogo de ações
UsuarioController, RegistroAcaoController, AcaoController, MissaoController, StatusController	controller	Endpoints REST
UsuarioRequestDTO, UsuarioResponseDTO, RegistroRequestDTO, RegistroResponseDTO, RegistroResultadoDTO, AcaoResponseDTO, MissaoResponseDTO, MissaoUsuarioResponseDTO	dto	Dados de entrada e saída (records)
CorsFilter	config	Libera o acesso do front-end (CORS)
SoulupApiApplication, ServletInitializer	raiz	Inicialização da aplicação Spring Boot

8. Procedimentos para rodar a aplicação

Ferramentas e versões

Ferramenta	Versão
Java JDK	21
Spring Boot	4.1.1 (Web MVC, Data JPA, Validation)
Maven	3.9 (incluído no projeto pelo Maven Wrapper `mvnw`)
IDE	IntelliJ IDEA (ou outra com suporte a Maven)
Banco de dados	Oracle (servidor da FIAP), driver ojdbc11
Documentação	springdoc-openapi 3.1.0 (Swagger UI)

Pré-requisito: banco de dados

A conta Oracle precisa ter as tabelas do SoulUp, criadas pelo script DDL de Building Relational Database, e as sequências e colunas de missões do script `SoulUp_SPRINT4.sql`, entregue na disciplina de Python.

Passo a passo no IntelliJ

- Baixe o projeto: `git clone https://github.com/matheusleite21/soulup-api.git`, ou extraia o .zip entregue.
- No IntelliJ, use **File > Open** e selecione a pasta `soulup-api`. O projeto é importado como Maven; aguarde o download das dependências.
- Em **Run > Edit Configurations**, na configuração `SoulupApiApplication`, preencha **Environment variables** com `DB_USER=seu_rm;DB_PASSWORD=sua_senha`.
- Execute a classe `SoulupApiApplication`. A API sobe em `http://localhost:8081`.
- Abra `http://localhost:8081/swagger-ui.html` para testar os endpoints.

Pelo terminal, a alternativa é: `./mvnw spring-boot:run` (Linux/macOS) ou `mvnw.cmd spring-boot:run` (Windows), com as variáveis `DB_USER` e `DB_PASSWORD` definidas.

Deploy na nuvem (Render)

O projeto possui um `Dockerfile` com duas etapas (compilação com Maven e execução com Java 21). No Render, foi criado um **Web Service** a partir do repositório no GitHub, com ambiente Docker, plano Free e as variáveis `DB_USER` e `DB_PASSWORD`. A cada push na branch `main`, o Render publica a nova versão. No plano gratuito o serviço "dorme" sem uso, e a primeira requisição pode levar cerca de um minuto.

9. Links da entrega

Item	Link
Repositório da API (GitHub)	https://github.com/matheusleite21/soulup-api
API publicada (Render)	https://soulup-api-k9nq.onrender.com
Swagger da API publicada	https://soulup-api-k9nq.onrender.com/swagger-ui.html
Repositório do front-end	https://github.com/matheusruiz-07/Challenge-1TDSPO